

Tim Farnung, Dieter Grünke, Erik Sauer, Leonhard Schneider

Projekt: StudyConnect

Lab2

Process Security Report: StudyConnect

Aktueller Prozess

Das Projekt wurde im Zuge des Faches Softwaretesting im agilen Ansatz entwickelt. Dies eignet sich weiterhin für das inkrementelle Umsetzen der Sicherheitsupdates im Zuge des Faches *Secure Software Delopment*.

Evaluation of the Development Process & Security Gaps (Prozessevaluierung & Lücken)

Aktueller Stand: Der aktuelle Entwicklungsprozess von StudyConnect nutzt bereits grundlegende CI/CD-Praktiken über GitHub Actions (wie in docs/09_ci_pipeline.md beschrieben). Dazu gehören das automatisierte Ausführen von Unit- und BDD-Tests (pytest, behave), Linting (Flake8, Black) via Pre-Commit-Hooks sowie Basis-Sicherheitsprüfungen mittels pip-audit, npm audit und einer SonarQube-Integration für Static Application Security Testing (SAST).

Identifizierte Sicherheitslücken im Prozess (Security Gaps): Obwohl Werkzeuge wie SonarQube Code-Smells und Basis-Fehler (wie debug=True oder offenes CORS) finden, hat unsere vorherige Analyse gezeigt, dass schwerwiegende logische Schwachstellen (wie IDOR, fehlende Autorisierung beim Bearbeiten von Tasks oder Gruppen-Beitritt) unentdeckt bleiben. Daraus ergeben sich folgende Prozesslücken:

Fehlendes Threat Modeling: In der Aufgabe davor wurden keine systematischen Bedrohungsanalysen (z.B. STRIDE) durchgeführt. Berechtigungskonzepte wurden nicht formal spezifiziert. Kein DAST (Dynamic Application Security Testing): Die API wird während der CI/CD-Pipeline nicht dynamisch auf Laufzeit-Schwachstellen z.B. auf unautorisierte Zugriffe getestet. Mangelhaftes Container- & Image-Scanning: Die Docker-Images (postgres, pgadmin, keycloak, eigene UI/App) werden vor dem Deployment nicht auf bekannte CVEs in den Basis-Images gescannt. Kein Secret Scanning im Code: Es gibt keinen automatisierten Prozess, der verhindert, dass versehentlich hartcodierte Credentials wie z.B. Keycloak-Secrets ins Repository gepusht werden.

Proposed Security Gates (Empfohlene Security Gates)

Um die identifizierten Lücken zu schließen, schlagen wir die Implementierung von mindestens vier verbindlichen "Security Gates" vor. Ein Pull Request darf nur gemerged werden, wenn alle Gates erfolgreich passiert wurden.

Gate 1: Hardened SAST & Secret Scanning Gate

Implementierung: Erweiterung der GitHub Actions um TruffleHog oder Gitleaks zum Aufspüren von Secrets, sowie Konfiguration eines strikten SonarQube "Quality Gates" (Fail bei neuen Vulnerabilities oder Security Hotspots). Rationale Begründung: Verhindert das versehentliche Einchecken von API-Keys, Datenbank-

Passwörtern oder Tokens (wofür .env-Dateien da sind). Das strikte SAST-Gate zwingt Entwickler, unsichere Code-Muster sofort zu beheben, anstatt technische Schulden aufzubauen.

Gate 2: Software Composition Analysis (SCA) & Container Scan Gate

Implementierung: Einsatz von Tools wie Trivy oder Docker Scout in der CI-Pipeline. Die Pipeline schlägt fehl, wenn in den gebauten Docker-Images (backend, ui) oder den NPM/Python-Abhängigkeiten Schwachstellen mit dem Schweregrad "Critical" oder "High" gefunden werden. Rationale Begründung: pip-audit und npm audit sind ein guter Anfang, decken aber das Betriebssystem des Docker-Containers z.B. veraltete OS-Pakete im Python-Image nicht ab. Dieses Gate stellt sicher, dass die Ausführungsumgebung sicher ist.

Gate 3: DAST & API Fuzzing Gate

Implementierung: Integration von OWASP ZAP (Zed Attack Proxy) Baseline Scan oder API-Security-Testsz.B. via Postman/Newman Security Collections gegen die laufenden Container in der Integrations-Umgebung der Pipeline. Rationale Begründung: Statische Code-Analyse versteht keine Geschäftslogik. Ein DAST-Tool sendet fehlerhafte oder bösartige Payloads an die API und prüft, ob die Endpunkte z.B. /api/tasks richtig absichern und keine sensiblen Daten leaken oder abstürzen.

Gate 4: Mandatory Security Peer Review Gate

Implementierung: GitHub Branch Protection Rules erfordern mindestens ein Code-Review von einem designierten "Head of Security" im Team, sobald Änderungen an auth.py, api.py oder services.py vorgenommen werden. Dazu wird eine verbindliche Checkliste genutzt. Rationale Begründung: Schwachstellen wie IDOR (Insecure Direct Object Reference) lassen sich oft nur durch menschliches Verständnis der Berechtigungslogik finden. Dieses Gate stellt sicher, dass Autorisierungsprüfungen niemals übersehen werden.

Security Aktivitäten in den SDLC Phasen

SDLC Phase	Security Aktivität	Methode/ Tool
1. Requirements	Threat Modeling	Erstellung eines STRIDE-Modells für neue Features, Anforderungen des CRA für das Projekt definieren
2. Design	Secure Architecture Review	Überprüfung der API-Endpunkte auf "Secure by Design", Anpassung d. Systemarchitektur
3. Implementation	Linting & Secret Scanning	Nutzung von Pre-commit-Hooks; Einsatz von TruffleHog/Gitleaks zur Verhinderung von hartkodierten Credentials.
4. Testing	SAST, SCA & DAST	SonarQube (SAST), Trivy (Container/SCA) und OWASP ZAP (DAST) in der CI-Pipeline.
5. Deployment	IaC Scanning	Überprüfung der Konfiguration (Dockerfiles, Kubernetes Manifests) auf Sicherheitslücken vor dem Build.
6. Maintenance	Vulnerability Monitoring	Kontinuierliche Überwachung der Abhängigkeiten; Logging/Auditing zur Erkennung aktiver Angriffe; Nutzung von

SDLC Phase	Security Aktivität	Methode/ Tool
CI/CD-Pipelines		

Governance Rules

Um zu verhindern, dass Security Gates den Entwicklungsprozess blockieren, während gleichzeitig die Sicherheit gewahrt bleibt, definieren wir folgendes "Break-Glass"-Protokoll. Policy: Security Gate Override & Emergency Exception

1. Zweck

Diese Regelung definiert die Bedingungen, unter denen Security Gates (SAST, SCA, DAST) umgangen werden dürfen, um den Betrieb sicherzustellen, ohne die Integrität von StudyConnect zu gefährden.

2. Autorisierte Rollen

Ein Override darf nur von folgenden Personen genehmigt werden:

- Head of Security: Autorisierung bei False-Positives oder nicht-kritischen Fehlern.
- Lead Architect/Projektleiter: Zwingend erforderlich bei "Critical/High"-Sicherheitswarnungen oder in Produktionsnotfällen.

3. Bedingungen für einen Override

Ein Override ist nur zulässig bei:

- False Positive: Ein Tool meldet eine Schwachstelle, die nach manueller Prüfung als nicht ausnutzbar identifiziert wurde.
- Operativer Notfall: Ein kritischer Produktionsfehler (z.B. Systemausfall) erfordert einen sofortigen Fix, und das Security Gate verhindert den Deployment-Prozess.
- Akzeptiertes Risiko: Eine bewusste Entscheidung des Managements, ein Risiko für einen definierten Zeitraum in Kauf zu nehmen, bis eine endgültige Lösung implementiert ist.

4. Post-Override Anforderungen

Wird ein Gate umgangen, sind folgende Schritte für die CRA-Compliance verpflichtend:

- Dokumentation: Ein "Security Exception Ticket" muss im Issue-Tracker (GitHub Issues) erstellt werden.
- Begründung: Das Ticket muss eine Risikoanalyse, den Grund für den Override und die Identität des Genehmigenden enthalten.
- Remediation-Plan: Es muss eine Folgeaufgabe mit einer maximalen Frist von 14 Tagen erstellt werden, um das zugrundeliegende Sicherheitsproblem zu beheben.
- Audit-Trail: Alle Overrides müssen in der Historie der CI/CD-Pipeline zur transparenten Nachverfolgung protokolliert sein.

CRA Compliance Assessment, Bewertung der Cyber Resilience Act Konformität

Der europäische Cyber Resilience Act (CRA) fordert für Produkte mit digitalen Elementen grundlegende Cybersecurity-Standards, insbesondere "Secure by default", den Umgang mit Schwachstellen und

Transparenz.

Einschätzung für StudyConnect:

Secure by Default & Design: Teilweise erfüllt. Die Applikation nutzt Keycloak für sichere Authentifizierung (Bearer Tokens statt Cookies, wodurch CSRF vermieden wird) und hat Debug-Modi für Produktion deaktiviert (SonarQube Fixes). Die festgestellten IDOR-Lücken verletzen jedoch das Prinzip "Secure by Design" massiv, da Zugriffe standardmäßig nicht strikt auf den Eigentümer beschränkt sind.

Vulnerability Handling: Verbesserungswürdig. Wir haben zwar Tools (pip-audit), die bekannte Schwachstellen melden, es fehlt aber ein dokumentierter Prozess, wie das Team kritische Fehler nach einem Release patcht und die Nutzer informiert. Es gibt keine Coordinated Vulnerability Disclosure (CVD) Policy.

Software Bill of Materials (SBOM): Nicht erfüllt. Aktuell generiert die CI-Pipeline keine maschinenlesbare SBOM z.B. im CycloneDX- oder SPDX-Format. Um CRA-konform zu sein, muss das Projekt jederzeit transparent ausweisen können, welche Drittanbieter-Bibliotheken (in welchen Versionen) verwendet werden. (Könnte leicht über ein Plugin wie syft in die GitHub Actions integriert werden).

Data Minimization: Weitgehend erfüllt. StudyConnect fragt nur essentielle Daten ab (Name, E-Mail, Universität). Es werden keine unnötigen Telemetriedaten gesammelt.

Fazit zur CRA-Compliance: Ohne die Generierung einer SBOM, das Schließen der Business-Logic-Schwachstellen (IDOR) und einen formalisierten Prozess zur Behandlung von Sicherheitspatches wäre das StudyConnect-Projekt in seiner aktuellen Form nicht CRA-konform und dürfte nach Inkrafttreten der Regulierung nicht auf dem europäischen Markt bereitgestellt werden.